



ORIENTAÇÃO À OBJETOS

Desenvolvimento Web II

INTRODUÇÃO

- A orientação à objetos é uma maneira de programar que modela os processos de programação de uma maneira próxima à realidade.
- Cada componente de um programa como um objeto, com suas características e funcionalidades.
- A POO também possibilita uma maior otimização e reutilização do código.

CLASSES

- Classe é a estrutura mais fundamental para a criação de um objeto.
- Uma classe nada mais é do que um conjunto de variáveis (propriedades ou atributos) e funções (métodos), que definem o estado e o comportamento do objeto.
- Quando criamos uma classe, temos como objetivo final a criação de objetos, que nada mais são do que representações dessa classe em uma variável.

CLASSE

```
class Noticia {  
    public $titulo;  
    public $texto;  
    function setTitulo($valor){  
        $this->titulo = $valor;  
    }  
    function exhibeNoticia(){  
        echo "<b>". $this->titulo . "</b><p>". $this->texto . "</p>";  
    }  
}
```

OBJETO

```
$not = new Noticia;  
$not->titulo = 'PHP OO';  
$not->texto = 'Este curso contém os seguinte tópicos: POO';  
$not->exibeNoticia();
```

HERANÇA

- Herança é uma forma de reutilização de código em que novas classes são criadas a partir de classes existentes.
- Será absorvido seus atributos e comportamentos, e complementando-os com novas necessidades.

HERANÇA

```
include_once('noticia.class.php');  
class NoticiaPrincipal extends Noticia  
{  
    public $imagem;  
    function exibeNoticia()  
    {  
        echo "<img src=\"". $this->imagem .\"\"><p> <b>". $this->titulo ."</b><p>" . $this->texto . "</p>";  
    }  
}
```

MÉTODO CONSTRUTOR

```
class Noticia
{
    public $titulo;
    public $texto;
    function __construct($valor_tit, $valor_txt){
        $this->titulo = $valor_tit;
        $this->texto = $valor_txt;
    }
}
```

Antes da palavra construct, são utilizados dos underline(__)

MÉTODO CONSTRUTOR NA CLASSE FILHA

```
<?php
include_once('noticia.class.php');
class NoticiaPrincipal extends Noticia{
    public $imagem;
    function __construct($valor_tit, $valor_txt, $valor_img)
    {
        parent::__construct($valor_tit, $valor_txt);
        $this->imagem = $valor_img;
    }
}
```

ENCAPSULAMENTO

- Este recurso possibilita ao programador restringir ou liberar o acesso às propriedades e métodos das classes.
- **Public** : Quando definimos uma propriedade ou método como public, estamos dizendo que suas informações podem ser acessadas diretamente por qualquer script, a qualquer momento. Até este momento, todas as propriedades e métodos das classes que vimos foram definidas dessa forma.
- **Protected** : Quando definimos em uma classe uma propriedade ou método do tipo protected, estamos definindo que ambos só poderão ser acessados pela própria classe ou por seus herdeiros, sendo impossível realizar o acesso externo.
- **Private** : Quando definimos propriedades e métodos do tipo private, só a própria classe pode realizar o acesso, sendo ambos invisíveis para herdeiros ou para classes e programas externos.

INTERFACES

- Interfaces permitem a criação de código que especifica quais métodos uma classe deve implementar, sem ter que definir como esses métodos serão tratados.
- Interfaces são definidas utilizando a palavra-chave `interface`, e devem ter definições para todos os métodos listados na interface.
- Classes podem implementar mais de uma interface se desejarem, listando cada interface separada por um espaço.

INTERFACE

```
interface iNoticia {  
    public function setTitulo($valor);  
    public function setTexto($valor);  
    public function exhibeNoticia();  
}
```

INTERFACE

```
include_once('noticia_interface.class.php');  
class Noticia implements iNoticia  
{  
    ...  
}
```

CLASSE ABSTRATA

- Classes abstratas são classes que não podem ser instanciadas diretamente, sendo necessária a criação de uma sub-classe para conseguir utilizar suas características.
- Isso não quer dizer que os métodos destas classes também precisem ser abstratos, isso é opcional.
- As propriedades não podem ser definidas como abstratas.

CLASSE ABSTRATA

```
abstract class Noticia
{
    protected $titulo;
    protected $texto;
    public function setTitulo($valor){
        $this->titulo = $valor;
    }
    abstract public function setTexto($valor);
    abstract public function exhibeNoticia();
}
```

PALAVRA CHAVE FINAL

- Classes definidas com a palavra-chave final não podem ser herdadas por outras classes.
- Quando um método é definido dessa forma, as subclasses que o herdarem não poderão sobrescrevê-los.

```
final class Noticia{  
    ...  
}
```


MÉTODOS E PROPRIEDADES ESTÁTICOS

```
class Noticia {  
    public static $nome_jornal = 'The Unicamp Post';  
    protected $titulo;  
    function setTitulo($valor) {  
        $this->titulo = $valor;  
    }  
    function exibeNoticia() {  
        echo "Nome do Jornal: <b>" . self::$nome_jornal . "</b><p><b>" . $this->titulo . "</b></p>";  
    }  
}
```

Para acessar o atributo/método fora do contexto da classe, devemos utilizar o nome da mesma e ::
echo "<p>" . Noticia::\$nome_jornal . "</p>";

MÉTODOS MÁGICOS

- `__set` : Este método pode ser declarado em qualquer classe e será executado toda vez que for atribuído algum valor à alguma propriedade do objeto. Ou seja, ele intercepta a atribuição de valores à propriedades de um objeto. Porém, para que este método funcione, estas propriedades devem estar definidas como `protected` ou `private`.
- `__get` : Este método pode ser declarado em qualquer classe e será executado toda vez que for solicitado o retorno do valor de alguma propriedade de um objeto.
- `__toString` : O método `__toString()` é chamado toda vez que se invoca a função `print` ou `echo` para um objeto.

MÉTODOS MÁGICOS

```
public function __get($key){  
    return $this->{$key};  
}
```

```
public function __set($key, $value){  
    $this->{$key} = $value;  
}
```

```
function __toString(){  
    return "<p>Classe <b>Noticia</b></p>";  
}
```